Estimating Collaborative Information to eliminate the need for retraining CF in LLMRecs

Your Name
Data Mining Lab
Dept. of CSE
Seoul National University

Overview

- LLM-based recommendation systems had relied solely on the text information of users and items for recommendations.

- Recently, approaches incorporating collaborative information have been increasing to enhance the performance of LLM-based recommendation systems.

- As a result, performance has significantly improved, but a **new issue has arisen**: when **new items** and **new users** are introduced, parts of the model need to be retrained.

Overview

- **This research firstly proposes** a process to estimate collaborative embedding, aiming to eliminate the need for **additional tuning** in LLM-based Recommendation when new items are added.

Outline

- **Intorduction**
- Proposed Method
- Contribution

LLM-based Recommenders

LLM-based recommendation systems demonstrate high performance in recommendation tasks by leveraging the reasoning knowledge of LLMs.

- **LLM-based recommendation systems** :
  Recommend items for user using **only text information.**

Information

User:
gender, age, preference..

Item:
Title, year, director ...

Prompt

• **Prompt template.** #Question: A user has given high ratings to the following items: ⟨*HisItemTitleList*⟩. Additionally, we have information about the user's preferences encoded in the feature ⟨*UserID*⟩. Using all available information, make a prediction about whether the user would enjoy the item titled ⟨*TargetItemTitle*⟩ with the feature ⟨*TargetItemID*⟩? Answer with "Yes" or "No". #Answer:

LLM

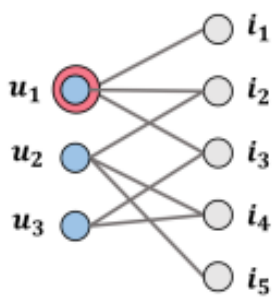
Yes or No

- However, since **only the content information** of users and items is utilized, items with similar content will produce the same recommendation results.

CF-Recommenders

CF Models

- A methodology for performing recommendations based on user-item interaction information.
- Graph, Matrix



User-Item Interaction Graph

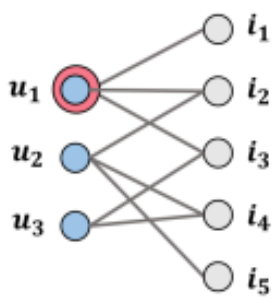
	i_1	i_2	i_3	i_4
u_1	1	2		2
u_2		4	3	
u_3			2	
u_4	2	4	1	

- Users and items embeddings are obtained from a **Graph or Matrix structure**.

CF-Recommenders

CF Models

- A methodology for performing recommendations based on user-item interaction information.
- Graph, Matrix



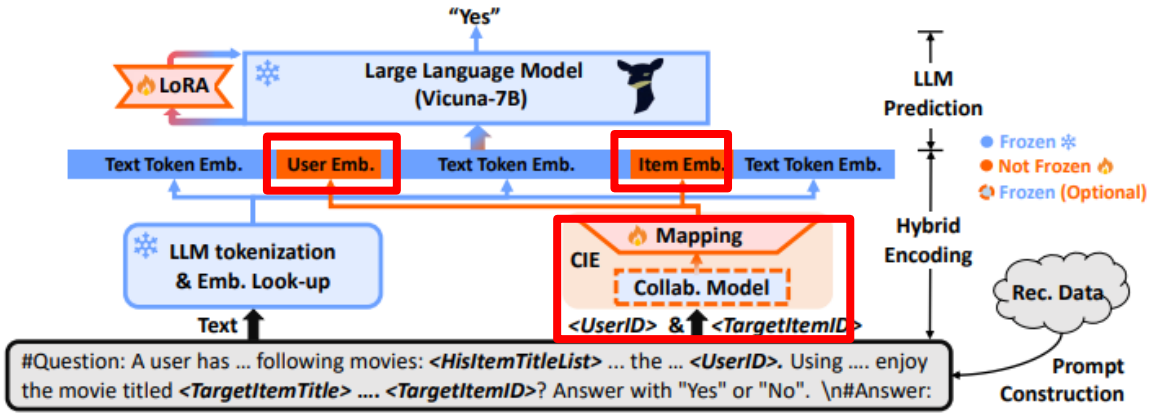
User-Item Interaction Graph

	i_1	i_2	i_3	i_4	i_5
u_1	1	2		2	3
u_2		4	3		4
u_3			2		2
u_4	2	4	1		1
u_5	5	2		3	1

- Users and items embeddings are obtained from a Graph or Matrix structure.
- When a new item is added, the embeddings for all items **must be retrained.**

Co-LLM

- Recently, LLM-based recommendation systems have been incorporating collaborative information from CF models to capture the user-item interaction.



When **new users and items** are introduced.

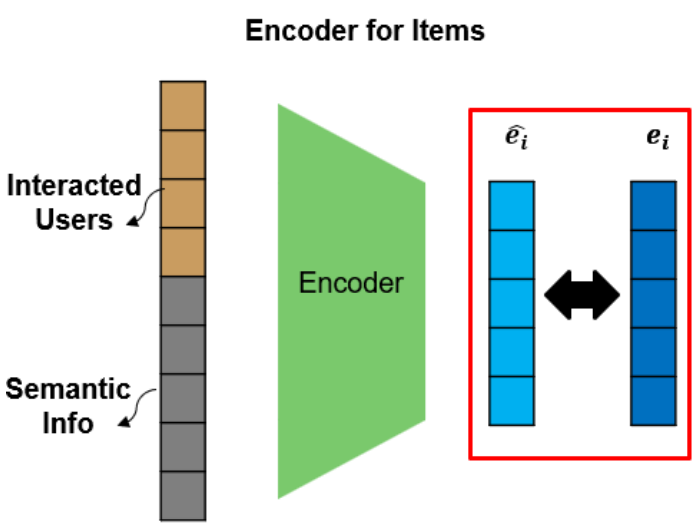
- The CF model must be retrained on the entire dataset.
- LLM's additional tuning process for the newly learned CF embeddings.

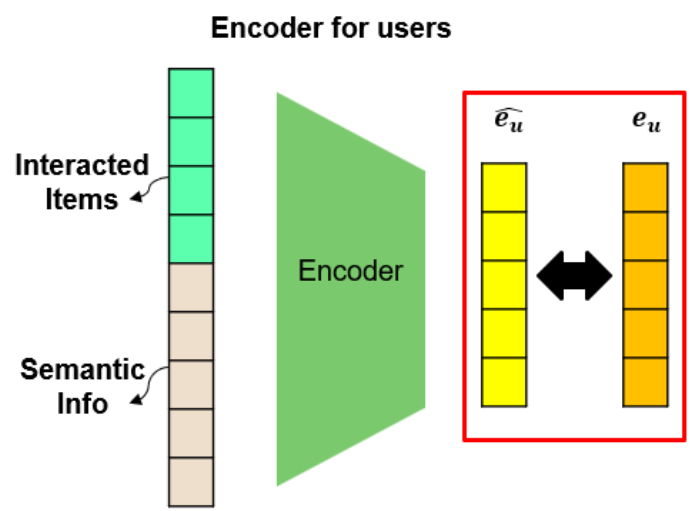
Architecture

This study propose 2 encoder structures that estimate collaborative information for new users and items.

- To prevent the retraining of CF models, when newly added items and users to the dataset.

$\boldsymbol{e_i}, \boldsymbol{e_u}$: embedding from pretrained CF model.
$\boldsymbol{\hat{e}_i}, \boldsymbol{\hat{e}_u}$: estimated e_i, e_u

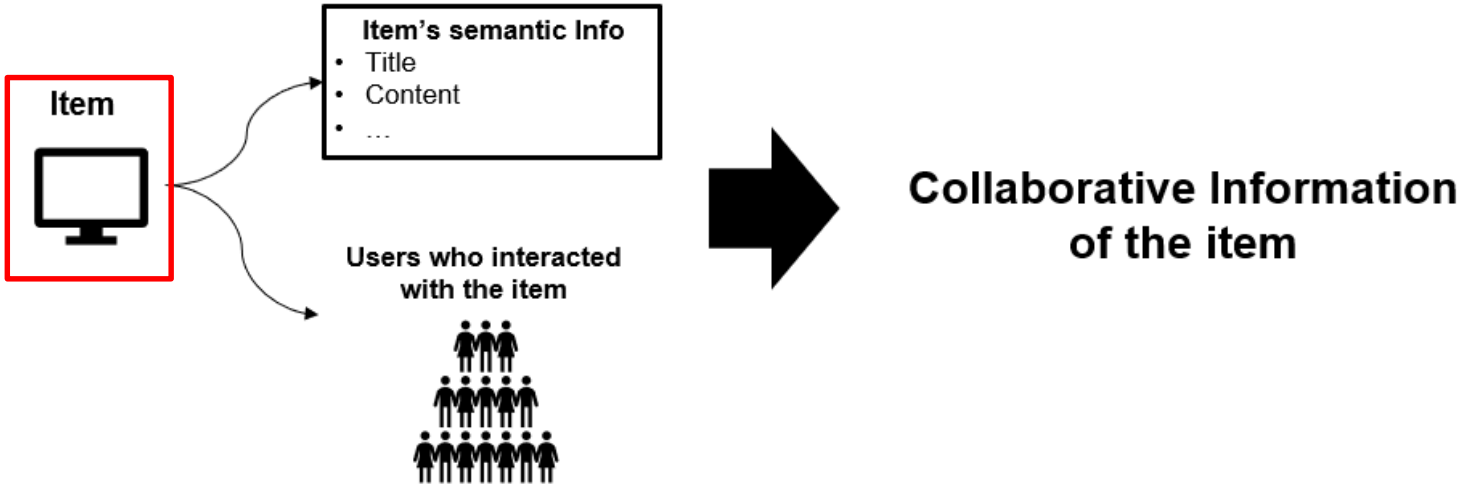




Hypothesis

To estimate collaborative information, two types of information are required.

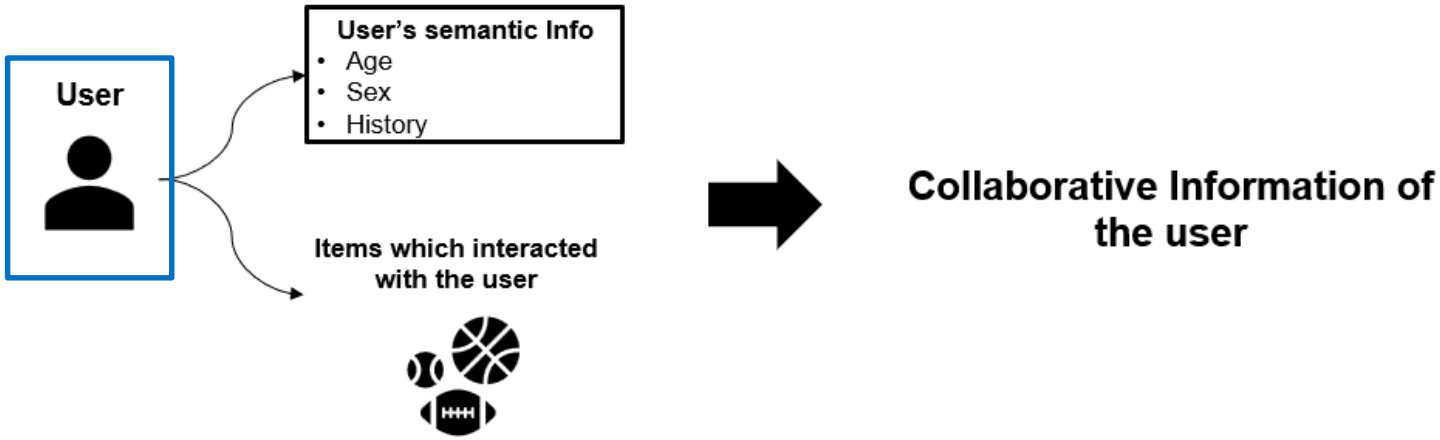
1. **The semantic information of the user or item to be estimated.**
   - To enable the model to recognize which user or item is given as input.
2. **Information on which users or items have interacted with.**
   - To provide the model with fundamental interaction information.



Hypothesis

To estimate collaborative information, two types of information are required.

1. **The semantic information of the user or item to be estimated.**
   - To enable the model to recognize which user or item is given as input.
2. **Information on which users or items have interacted with.**
   - To provide the model with fundamental interaction information.



Encoder for items

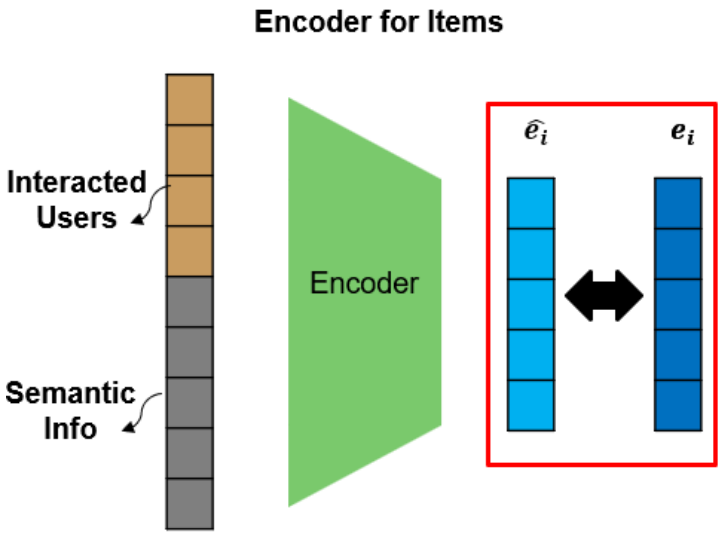
Encoder for Items estimate collaborative embeddings of the items.

Input:

- Semantic Information of an item
- The information on users who interacted with the item.

Output:

- Estimated collaborative embedding of the item.



Encoder for users

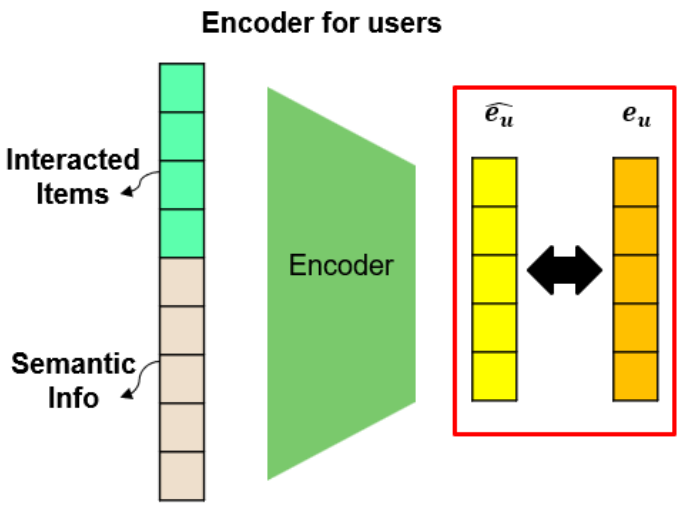
Encoder for users estimate collaborative embedding of the users.

Input:

- Semantic Information of an user
- The information on users who interacted with the item

Output:

- Estimated collaborative embedding of the user



Problem Definition

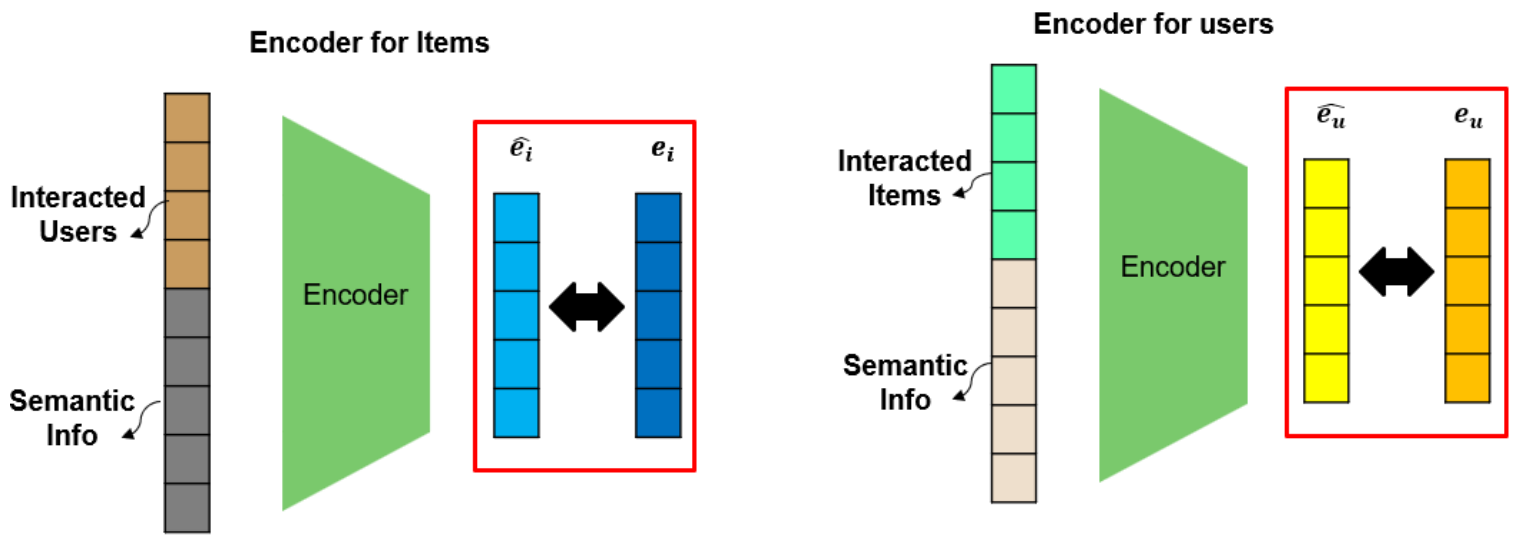
- **Given:**
  - Pretrained CF Model
  - Semantic Information of users and items
  - User-item interaction

- **To train 2 functions:**
  - A function generate **new item‘s** collaborative information.
  - A function generate **new user’s** collaborative information.

Outline

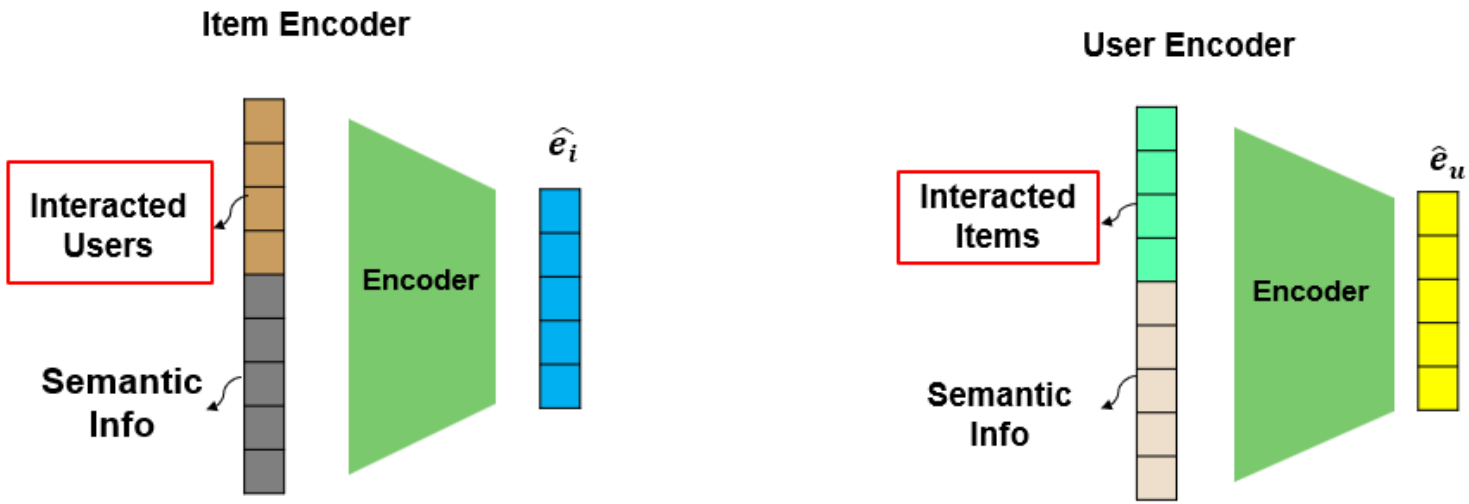
- Introduction
- **Proposed Method**
- Contribution

Overview



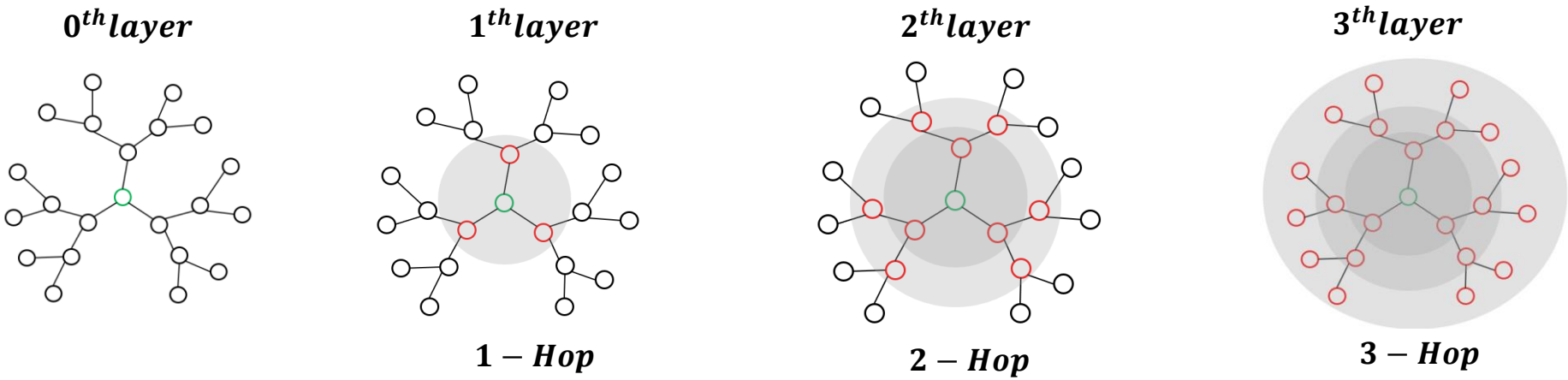
- I propose 2 encoder models that estimates CF model embeddings based on semantic and interaction information.
- One for Items CF embeddings.
- Another for users CF embeddings.

Overview



- Since the inputs of both encoders need to represent which users and items have interacted.
- So a method is required to densely represent both item and user information.

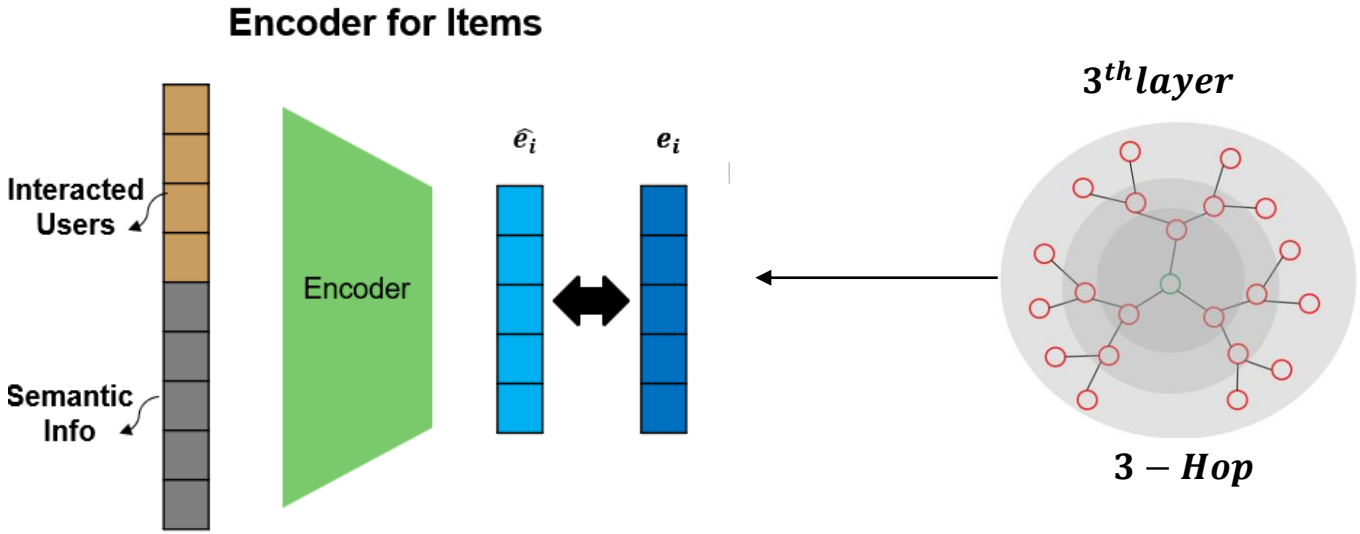
Overview



A progressive encoding process will be necessary to estimate CF embeddings.

- GNN gradually incorporates information from the closest nodes as it passes through each layer.
- However, the proposed encoder may struggle to directly estimate the embedding of the kth layer.

Overview



A progressive encoding process will be necessary to estimate CF embeddings

- GNN gradually incorporates information from the closest nodes as it passes through each layer.
- However, the proposed encoder may struggle to directly estimate the embedding of the kth layer.

Challenges

- **[C1]** How to represent all users and items for interaction information.

  [I1] Hash Embedding: Users and items are represented as the sum of embeddings corresponding to multiple hash values.

- **[C2]** How to integrate 2 different types embeddings (semantic and interaction embedding)?

  [I2] Aligning 2 different types of embedding to the same vector space.

- **[C3]** How to train the encoder to progressively integrate each hops information?

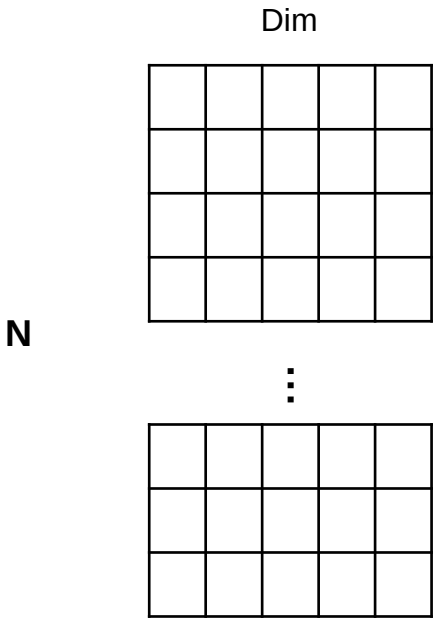
  [I3] Multi-Hop Tuning: Multiple encoders are trained to minimize the difference with each layer of the CF model.

Hash Embedding(2017,NIPs)

"Hash embedding" refers to an efficient representation of large-sized categorical values, such as words, users and items by utilizing multiple hash functions.

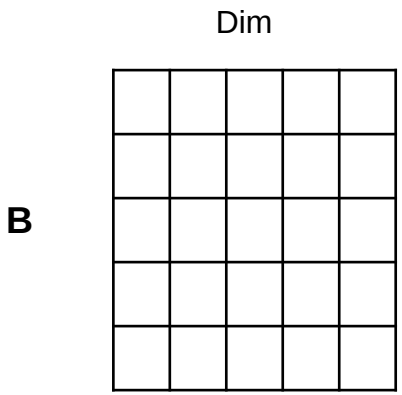
N: # of users or words..
B: bucket size of hash functions

Naïve Methods



Large Embedding Table

Hash Embedding



Smaller Embedding Table

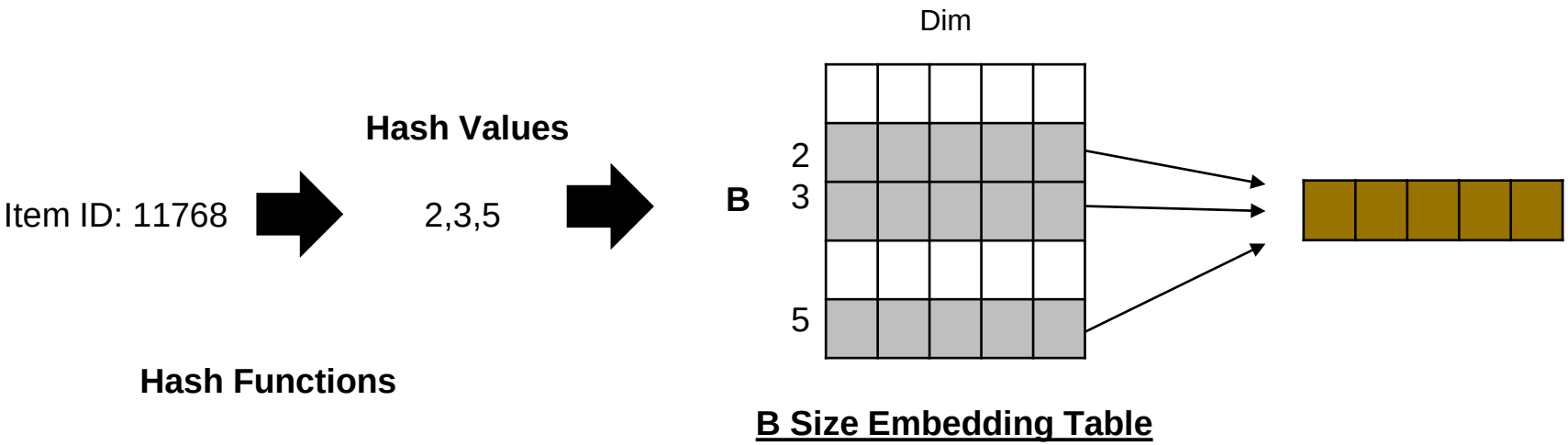
Hash Embedding(2017,NIPs)

1. Apply **k** hash functions with the same bucket size to the unique ID of a user or item.

2. Use the **k** hash values as indexes to look up embeddings from the embedding table of bucket size.

3. Sum all the retrieved embeddings.

B: bucket size of hash functions

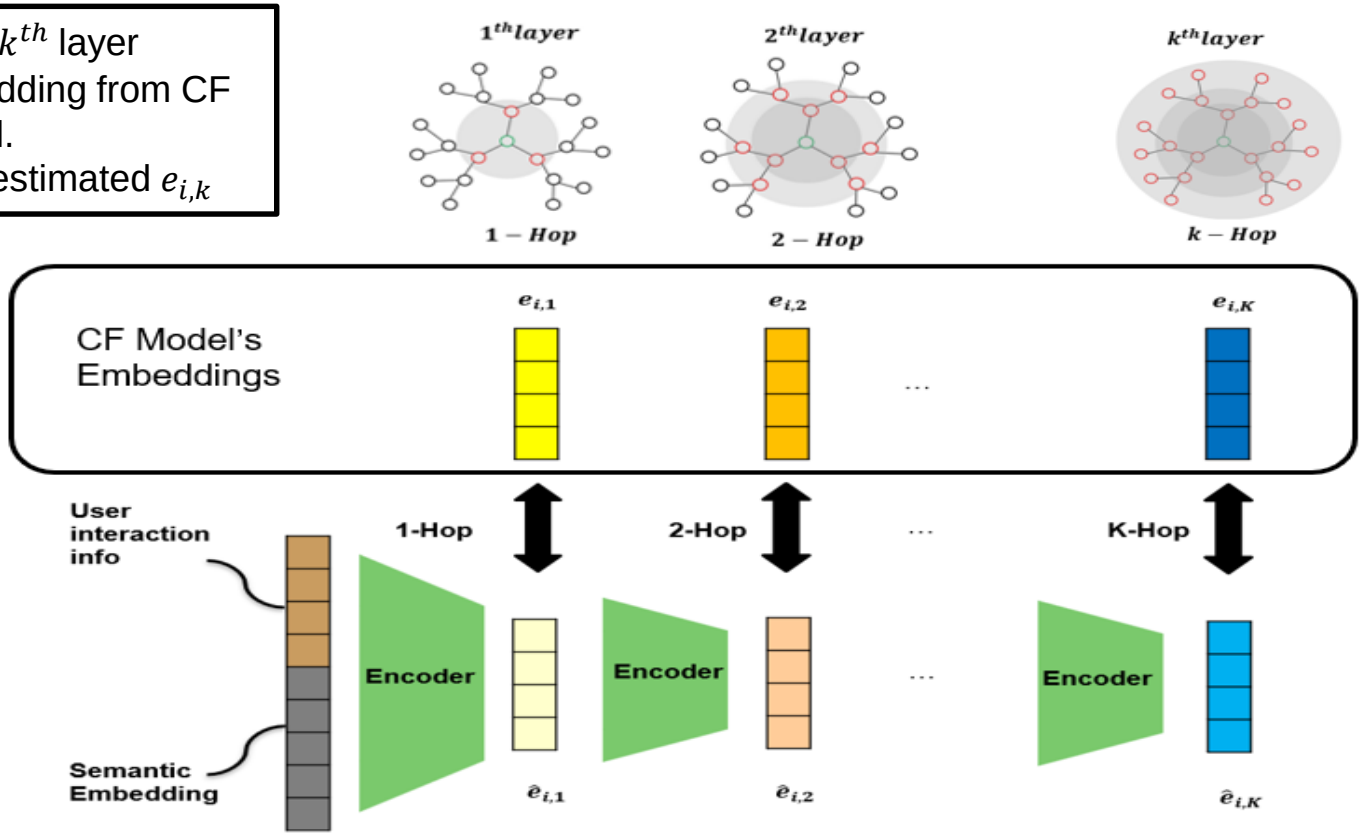


Aligning

Multi-Hop Tuning

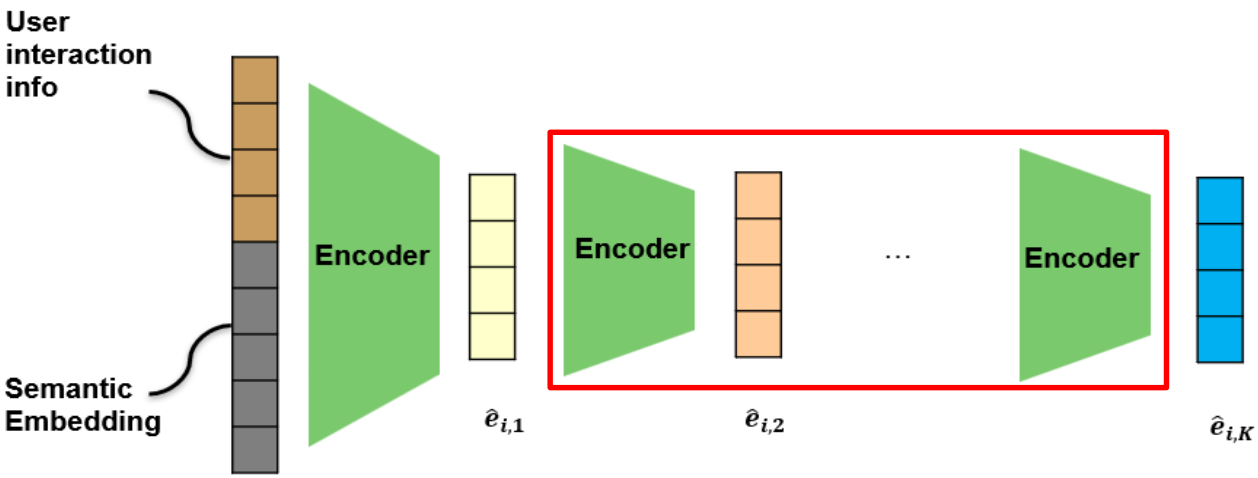
Multiple encoders are designed to progressively reduce the difference between each layer of the CF model and the corresponding encoder during training.

$\boldsymbol{e_{i,k}}$: k^{th} layer embedding from CF model.
$\hat{\boldsymbol{e}}_{\boldsymbol{i,k}}$: estimated $e_{i,k}$



Multi-Hop Tuning

- Each layer-specific encoder learns what information should be captured at that level.
- The encoder structure becomes similar to GNNs, which perform **stepwise aggregation**, potentially improving CF embedding estimation.



Outline

- Problem Definition
- Proposed Method
- **Conclusion**

Contribution

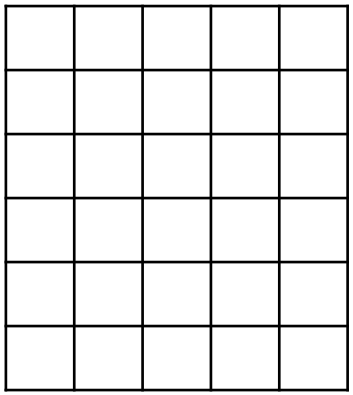
- This approach aims to firstly estimate the CF model's embeddings.
- By estimating the original embeddings of the CF model, the need for retraining the CF model is eliminated.
- Additionally, since the well-tuned CF model embeddings are used for LLM, there is no need for joint tuning of the LLM and CF model.

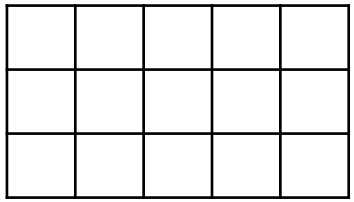
Thank you !

Embeddings for interaction information

- **An additional trainable embedding table** is allocated for new items.
- **During training**: the model simulates the addition of new items by randomly assigning some items to the additional embedding table.
- **During testing:** when a new item is introduced, it is assigned to the additional embedding table to construct interaction information.

Original Embedding table for items



Both of them are trainable

Additional Embedding for new items